```
        .style("background", "rgba(255,255,255,1)")
        .style("border-color", "rgba(0,0,0,1)");

  var selectedNode = d3.selectAll("img")
              .filter(function(d) {return d.x == selectValue}).node();

  selectedNode.parentNode.appendChild(selectedNode);

  d3.select("div.gallery").transition().duration(2000)
        .tween("scrollTween", scrollTopTween(scrollTarget));

  function scrollTopTween(scrollTo) {
    return function() {
        var i = d3.interpolateNumber(this.scrollTop, scrollTo);
        return function(t) { this.scrollTop = i(t); };
    };
  };
};

d3.select("div.gallery").style("height", "50%")
          .style("overflow","scroll").style("border", "2px black solid");

d3.select("#traditional").append("select")
    .on("change", zoomTo)
    .selectAll("option")
    .data(d3.selectAll("img").data()).enter()
    .append("option")
    .attr("value", function(d) {return d.x;})
    .html(function(d) {return "Image #" +d.x;});
```

Zooms that image in the same way we did with mouseover

Brings that image forward in the DOM

Scrolls the div with a tween

Continuously updates the div's scrollTop attribute to be a value between its current scrollTop and the calculated location of the selected image

This produces a gallery like that in figure 8.11. If we wanted to deploy this gallery, we'd need to do some cleaning up. But the purpose of this chapter is to demonstrate how you can use D3 functionality to work with bitmaps, divs, and other traditional materials of web design. Notice that we'll need to adjust some of our workflows and syntax and also integrate CSS more, but buttons, images, and paragraphs can be

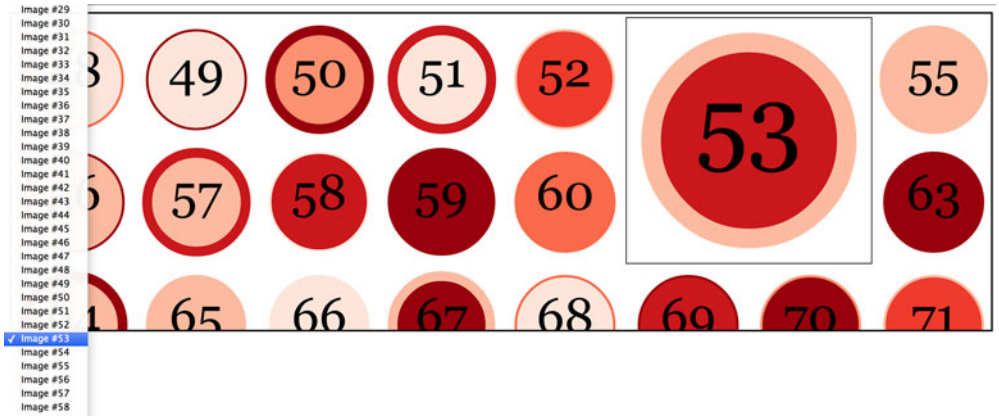


Figure 8.11 Selecting an image from the list scrolls the div to the proper location and increases its size.